

1. Alice runs a Set-UID program that is owned by Bob. The program tries to read from `/tmp/x`, which is readable to Alice, but not to anybody else. Can this program successfully read from the file?

No. The Set-UID program owned by Bob will try to read `/tmp/x` as Bob, and not Alice. And since only Alice can access that file, the program cannot.

2. A process tries to open a file for read. The process's effective user ID is 1000, and real user ID is 2000. The file is readable to user ID 2000, but not to user ID 1000. Can this process successfully open the file?

No. Since the effective user ID doesn't have access to the file, the process cannot open it.

3. A root-owned Set-UID program allows a normal user to gain the root privilege while executing the program. What prevents the user from doing bad things using the privilege?

Good programming practice while writing the set-aid program. By not mixing user input with trusted data, and not using vulnerable APIs such as `system`, the root program will be very limited in scope as to what it can do.

4. We are trying to turn a program `prog` owned by the `seed` user into a Set-UID program that is owned by `root`. Can running the following commands achieve the goal?

```
$ sudo chmod 4755 prog
$ sudo chown root prog
```

No. Upon `chown`, or even file copy, the Set-UID bit is reset. So running `chmod 4755` should always be the last step in the set up.

5. The `chown` command automatically disables the Set-UID bit, when it changes the owner of a Set-UID program. Please explain why it does that.

The decision to make a program a Set-UID one is taken by the owner of the program. Since `chown` switches ownership, it's now the job of the new owner to decide whether or not the program will always run with his privileges.

6. When we debug a program, we can change the program's internal variables during the execution. This can change a program's behavior. Can we use this technique to debug a Set-UID program and change its behavior? For example, if the program is supposed to open the `/tmp/xyz` file, we can modify the filename string, so the Set-UID program ends up opening `/etc/passwd`.

No. Debugging a Set-UID program requires running the debugger as root. Otherwise anyone can run the debugger on the `/bin/su` program and run everything with root privileges.

7. Both `system()` and `execve()` can be used to execute external programs. Why is `system()` unsafe while `execve()` is safe?

If the external program is decided by the external factors, such as user input, since `system()` uses `/bin/sh` internally, the user can input a dummy program, followed by a `;` and include potentially malicious commands after that and they'll be executed too. With `execve()`, the whole thing(;
malicious command after) will be treated as a single program and will therefore fail to execute.

8. When a program takes an input from users, we can redirect the input device, so the program can take the input from a file. For example, we can use `prog < myfile` to provide the data from `myfile` as input to the `prog` program. Now, if `prog` is a root-owned Set-UID program, can we use the following method to get this privileged program to read from the `/etc/shadow` file?

```
$ prog < /etc/shadow
```

No. The fact that `prog` is a Set-UID program with root privileges has nothing to do with the file that's input after. Since a normal user can't access `/etc/shadow`, he/she can't use it to redirect the input device. Trying to do so will result in a permission denied error.

9. When a parent Set-UID process (effective user ID is root, and the real user ID is bob) creates a child process using `fork()`, the standard input, output, and error devices of the parent will be inherited by the child. If the child process drops its root privilege, it still retains the access right to these devices. This seems to be a capability leaking, similar to what we covered in Chapter 1.4.4. Can this pose any danger?

No. Though the parent Set-UID process runs as root, the parent of the parent Set-UID process has a user ID of bob. This is because Bob originally executed the parent Set-UID program from a shell or similar. Therefore, the standard input, output, and error drives of the parent Set-UID program were inherited from the shell, which is then inherited again by the grandparent process. Since they have the same privileges, there is no capability leak.